AMITY UNIVERSITY

A RESEARCH & INNOVATION DRIVEN UNIVERSITY GRADE 'A' ACCREDITED BY NAAC

A

PROJECT REPORT

ON

"TogetherHub – Unified space for ideas, code, and conversations"

SUBMITTED

IN

PARTIAL FULFILLMENT

OF

THE REQUIREMENTS FOR AWARD OF DEGREE

OF

Master of Computer Applications

By

Abhinav Pandey **Enroll. No: A809145024017**

Under the Guidance of

Dr. Shikha Tiwari

DEPARTMENT OF INFORMATION & TECHNOLOGY

AMITY INSTITUTE OF INFORMATION AND TECHNOLOGY

AMITY UNIVERSITY CHHATTISGARH

RAIPUR- 493225

Batch 2024- 2026

Amity Institute of Information and Technology

Department of Information & Technology

CERTIFICATE

This is to certify that the Project Entitled

TogetherHub – Unified space for ideas, code, and conversations

Submitted by

Abhinav Pandey **Enroll. No: A809145024017**

is a bonafide work carried out by students under the supervision of **Dr. Shikha Tiwari** and it is submitted towards the partial fulfillment of the requirement of Bachelor of Computer Applications Dissertation Project.

Dr. Shikha Tiwari
Internal Guide
Dept. of Computer Science & Engineering

Dr. Shikha Tiwari
Project Guide
Dept. of Computer Science & Engineering

Prof. (Dr.) Vinay Kumar Singh
Deputy Director
Amity School of Engineering & Technology

Signature of Internal Examiner **Signature of External Examiner**

DECLARATION

I, the undersigned, hereby declare that the work presented in this Project, titled " TogetherHub – Unified space for ideas, code, and conversations ", is our original work. It has been completed as part of our academic requirements and has not been submitted elsewhere for any degree, diploma, or other qualification.

I affirm that the content of this work is the result of my own efforts, except where explicitly acknowledged through proper citations and references.

I understand that any form of plagiarism or unauthorized use of external sources without proper acknowledgment may result in academic penalties as per the guidelines of Amity University Chhattisgarh.

Name	Enrollment No.	Signature	Date
Abhinav Pandey	A809145024017	_____	

ACKNOWLEDGMENT

It gives me immerse pleasure in presenting the project report on **'TogetherHub – Unified space for ideas, code, and conversations'.**

I would like to take this opportunity to thank my internal guide and project guide **Dr. Shikha Tiwari** for giving us all the help and guidance I needed. I am grateful for his kind support. His all-valuable suggestions were very helpful.

I am also grateful to **Prof. (Dr.) Vinay Kumar Singh**, Deputy Director of ASET and AIIT, for their indispensable support & suggestions.

In the end my special thanks to **all the teachers and staff** for providing various resources such as a laboratory with all needed software platforms, continuous Internet connection, for my Project.

Abhinav Pandey
(MCA 2024-26)

Plan of Project Execution

Week	Month	Description	Date	Duration
Week-1	July	Idea finalized, objectives set, CRDT & WebSocket research, MVP & tech stack defined, structure/scripts created.	21/07/25 – 25/07/25	5 days
Week-2	July	Express + React setup, Socket.IO connection tested, DB schema made, Auth UI + JWT backend done.	28/07/25 – 01/08/25	5 days
Week-3	August	JWT auth added, protected routes, Room dashboard + APIs, Socket.IO rooms broadcast tested.	04/08/25 – 08/08/25	5 days
Week-4	August	Monaco editor added, settings configured, Y.Doc + y-websocket + y-monaco binding integrated.	11/08/25 – 15/08/25	5 days
Week-5	August	Cursor presence (Awareness), Y.Doc persist/load, sync conflicts debugged, editor module finished.	18/08/25 – 22/08/25	5 days
Week-6	August	Whiteboard(canvas) built, drawing tools/events added, Y.Array sync + re-render logic done.	25/08/25 – 29/08/25	5 days
Week-7	September	Whiteboard presence + DB save/load, zoom/pan/clear added, unified split/tab UI verified.	01/09/25 – 05/09/25	5 days
Week-8	September	Code→Whiteboard DnD, snippet block added, Real-time Chat panel/events + presence unified.	08/09/25 – 12/09/25	5 days
Week-9	September	Modules integration tested, UI/UX bugs fixed, errors notified, real-time lag optimized.	15/09/25 – 19/09/25	5 days
Week-10	September	Shapes + Text tool(Y.Text) added, select/move/resize/delete board logic + AI code explain MVP made, feedback noted.	22/09/25 – 26/09/25	5 days

Week-11	Oct	WebRTC started, Socket.IO signaling, P2P calls tested, UI responsiveness improved.	29/09/25 – 03/10/25	5 days
Week-12	October	Tests done, frontend/backend deployed, WSS added, deploy bugs fixed, env doc written.	06/10/25 – 10/10/25	5 days
Week-13	October	Manual + report draft started, architecture/Yjs diagrams added, terminology checked.	13/10/25 – 17/10/25	5 days
Week-14	October	Load/sync tested, screenshots added, CRDT vs OT analysis written, slides UI started.	27/10/25 – 31/10/25	5 days
Week-15	November	Slides made, diagrams/roadmap added, demo + Q&A practiced, mentor feedback applied.	03/11/25 – 07/11/25	5 days

Table 1 : Plan of Project Execution

ABSTRACT

Together Hub is a unified, web-based real-time collaboration platform conceptualized to bridge the gap between fragmented digital teamwork tools by providing a synchronized space for ideas, software development, visual brainstorming, and contextual communication. In an era where remote work, hybrid learning, and geographically distributed teams dominate academic and professional workflows, the demand for a low-latency, conflict-free, secure, and highly extensible collaboration system has become critical. Existing platforms offer partial support—either focusing on document co-editing, UI design, or communication—but lack a truly unified multi-modal environment that seamlessly integrates text, code, whiteboard interactions, and live messaging without proprietary dependency or resource overhead.

To address this, Together Hub employs modern CRDT (Conflict-Free Replicated Data Type) algorithms using Yjs, enabling concurrent edits across distributed nodes while maintaining eventual consistency without central lock dependency. Socket.IO WebSockets form the backbone of live update propagation, providing ultra-responsive bi-directional communication between users and collaboration rooms. The platform integrates the Monaco Code Editor for real-time programming collaboration, configured to sync code changes, cursor presence, and awareness states among connected peers. Additionally, an interactive Canvas-Based Whiteboard Module is implemented using Yjs Y.Array structures, supporting collaborative drawing, annotations, shapes, and state persistence. A dedicated Room-Based Real-Time Chat Panel, powered by Socket.IO events, enables instant contextual discussions parallel to live editing sessions, improving team awareness and cognitive flow.

Security is ensured through JWT-based authentication, along with protected routing and secure session life-cycle management—creation, joining, broadcast states, exit handling, and termination. The backend architecture built using Node.js with Express delivers REST APIs, socket room events, persistence logic, and signaling for planned WebRTC-based voice/video P2P calls. MongoDB is used as the primary database to store user data, room metadata, document snapshots, awareness states, whiteboard objects, and chat histories, supporting scalability for high concurrency loads.

Together Hub is engineered with a modular and future-ready plugin architecture, enabling easy integration of forthcoming capabilities such as Shared Terminal Collaboration, AI-Assisted Code Explanation, intelligent auto-suggestions, tracking analytics dashboards, session persistence, and rollback versioning without system bloating. The frontend UI, constructed using React.js, maintains full responsiveness, smooth transitions, split-screen collaboration view, structured layouts, and optimized update-handling for low-end devices and constrained network environments. Comprehensive unit, integration, and load testing with simulated and real users validates synchronization robustness, performance reliability, feature integrity, and user adoption readiness.

The project aims to revolutionize interactive collaboration by ensuring seamless synchronization, unified user presence indicators, encrypted communication, scalable cloud deployment, low-latency updates, conflict-free user interactions, open-source flexibility, and academic/startup-friendly usability. Together Hub not only fulfills present-day real-time collaboration needs but also lays a strong groundwork for continuous enhancement aligned with emerging AI-driven distributed system paradigms, empowering teams to collaborate smarter, faster, and more securely.

Table of Contents

S.No	Content	Page No.
1.	Introduction	
1.1	Background and Motivation	
1.2	Problem statement	
1.3	Objectives	
1.4	Scope of the Project	
1.5	Methodology Overview	
1.6	Organization of the report	
2.	Literature Review	
2.1	Overview of existing solutions	
2.2	Research gaps and relevant studies	
2.3	References (APA style)	
3.	System Architecture	
3.1	Introduction to system design	
3.2	System architecture	
3.3	Flowchart	
3.4	Use case diagram	
3.5	Technology stack	
4.	Design and Implementation	
4.1	Overview	
4.2	Frontend implementation	
4.3	Backend implementation	
4.4	UI designs and visuals	
4.5	Code snippets	
5.	Conclusion and Future scope	
6.	References	

List of Figures:

Figures	Page No.
3.2 System architecture	
3.3 flowchart	
3.4 use case diagram	
Fig. 4.4.a – Login / Sign up Page	
Fig. 4.4.b - Dashboard	
Fig. 4.4.c – Rooms page	
Fig. 4.4.d – Document editor page	
Fig. 4.4.e – Code Editor page	
Fig. 4.4.f – Whiteboard page	
Fig.4.5.a – login code	
Fig.4.5.b – Home section code	

Chapter – 1

INTRODUCTION

1. Background and Motivation

The evolution of digital collaboration has accelerated dramatically in the post-pandemic era, where virtual teamwork is no longer optional but a foundational requirement across education, industry, and innovation-driven environments. Real-time collaborative platforms have enabled distributed participation, yet most solutions remain fragmented, proprietary, or limited by scalability bottlenecks. Tools that allow shared editing of documents often lack integrated coding or interactive ideation capabilities, while code-centric platforms struggle with visual collaboration or instantaneous peer communication, forcing teams to adopt multiple tools that disrupt cognitive continuity and productivity. Additionally, platforms constructed for enterprise-grade environments often become resource-heavy, creating accessibility barriers for students, startups, or users operating on low-bandwidth networks and non-high-performance devices.

The inspiration for the platform is guided by real-world complexity: academic project teams struggling with file conflicts, developers sharing snippets over chat without live editing, mentors lacking presence cues during remote sessions, and startups unable to afford costly tools but needing robust real-time syncing. By combining technologies like Yjs CRDT, WebSockets via Socket.IO, a powerful Monaco-based collaborative editor, and an interactive canvas whiteboard synced through distributed document states, Together Hub aims to empower users through a unified shared workspace. The goal is to deliver a platform that is frictionless, developer-friendly, lightweight for constrained environments, and scalable for future-ready integrations such as collaborative terminals, rollback versioning, and enhanced media interaction through WebRTC signaling. Thus, Together Hub is not a standalone tool but a research-driven prototype pushing toward open, scalable, and synchronous teamwork infrastructure.

Key motivations include:

- Unifying multi-modal teamwork into one synchronized space.
- Removing dependency on closed proprietary ecosystems.
- Reducing sync conflicts using decentralized CRDT document states.
- Providing low-latency collaboration for constrained devices/networks.
- Enabling contextual communication parallel to live editing.
- Encouraging future expansion into AI-assisted collaborative modules without system bloat.
- Creating an academic-level tool suitable for group projects, peer learning, mentor-led sessions, and startup-driven development environments.

1.1 PROBLEM STATEMENT

Current collaboration platforms fall short in delivering a truly unified environment that supports real-time document editing, concurrent code collaboration, and collaborative whiteboard brainstorming alongside built-in contextual communication. Most systems are either:

- Too resource-heavy to scale efficiently on low-end devices or limited internet bandwidth,
- Confined to proprietary ecosystems, resulting in vendor lock-in, or
- Fragmented by design, forcing users to switch between multiple tools for chat, code, and visual ideation, which interrupts workflow continuity.

These gaps lead to synchronization conflicts, communication delays, high dependency costs, poor adaptability, and reduced user productivity. Therefore, there is a pressing need for a modular, lightweight, secure, and synchronized platform enabling seamless multi-modal collaboration without performance loss or integration barriers.

1.2 OBJECTIVES

The primary objectives of the Together Hub project are:

- To design and implement a real-time collaboration platform that supports simultaneous multi-user editing of documents, code, and canvas-based drawings with near-zero latency.
- To provide live presence awareness using cursor positions, typing indicators, and active room states to enhance remote coordination and reduce merge conflicts.
- To integrate contextual communication tools including real-time chat and room-based broadcasting within the same workspace to enable synchronized discussions.
- To incorporate secure authentication and access control using industry-standard protocols like JWT to improve visitor privacy, data integrity, and authenticated workspace usage.
- To design a scalable backend architecture enabling seamless cloud deployment for high concurrency loads.
- To ensure modular extensibility for future upgrades like AI assistance, collaborative terminal sessions, version rollback, and WebRTC-based media communication without system bloat.

1.3 SCOPE OF THE PROJECT

The scope of Together Hub spans multiple layers of academic-to-industry relevance, focusing on multi-modal synchronous collaboration while prioritizing extensibility, scalability, and security. The project currently targets MVP (Minimum Viable Product) readiness while architecting expansion layers for future research innovation.

Included within scope:

- **Decentralized real-time document synchronization** using Yjs CRDT (Y.Doc + Y.Text + Y.Array).
- **Room-based multi-user concurrent editing sessions** using Socket.IO room broadcasting.
- **Collaborative code development environment** using the Monaco Code Editor bound to Yjs states.
- **Canvas-whiteboard spatial ideation** synced through distributed document updates.
- **Presence & awareness tracking** using cursor movement, typing state, and editing activity indicators.
- **Integrated chat & message broadcast within rooms** using real-time Socket.IO events.
- **Secure routing & session tracking** using token authentication via JWT.
- **Cloud-scalable backend prototype** with document snapshots, chat histories, and metadata persistence using MongoDB.

Planned future expansion (architected but not currently active):

- AI-powered code explanation & smart suggestions for selected snippets.
- Collaborative terminal sessions using CRDT-driven shared command flows.
- WebRTC-based peer-to-peer voice/video calls enabled through Socket.IO signaling.
- Room-based notifications, role-based permissioning, real-time analytics, version rollback, plugin-driven enhancements.
- Multi-media support for submitting documents, idea boards, project snapshots in rooms.

Out of scope / limitations:

- The platform currently supports MVP collaboration but not full enterprise load.
- Live AI auto-completion is limited and provided only as prototype code explanation.
- Persistent long-term offline storage of edits beyond snapshots is planned but not fully deployed.
- Collaborative terminal is architected but not yet implemented.
- Public large-scale user deployment is not current target; platform is in academic-prototype phase.

Together Hub is a scalable research-driven prototype intended for academic teams, startup developers, and remote workflows emphasizing low-latency CRDT synchronization, cognitive unification, and modular extensibility.

1.4 METHODOLOGY OVERVIEW

The development methodology of Together Hub follows modular and test-driven execution, including:

- **Phase 0: Project Setup & Tech Stack Finalization**
  - Project structure creation (client/server),
  - Initialization of scripts, configurations, and architecture planning.
- **Phase 1: Authentication Module**
  - Login/registration UI built on React,
  - Password hashing & JWT generation in backend,
  - Token storage, protected routes, secured session lifecycle.
- **Phase 2: Collaboration Rooms Module**
  - Room dashboard UI,
  - REST APIs for room creation/joining/fetching,
  - Socket.IO room logic for broadcast states.
- **Phase 3: Code Editor Module**
  - Monaco editor integration,
  - Y.Doc initialization,
  - y-websocket provider configured with server,

    - y-Monaco bindings for conflict-free co-edit,
    - Cursor presence using Awareness API.
- **Phase 4: Whiteboard Module**
    - Built using HTML5 Canvas,
    - Drawing tools synced using Yjs Array structures,
    - DB persistence, re-render listeners, zoom/pan/clear.
- **Phase 5: Chat Module**
    - Dedicated chat panel within rooms,
    - Messaging events (send/receive),
    - Presence awareness indicators,
    - Basic error notifications.
- **Phase 6: Deployment & Testing**
    - Unit, integration, and concurrency testing,
    - Backend deployed to cloud host, frontend to CDN or hosting,
    - WSS configured, deployment bugs debugged.
- **Extensibility Layer**
    - Plugin design for future AI assistance, terminal collaboration, version rollback, analytics, and WebRTC signaling.

1.5 ORGANIZATION OF THE REPORT

The report is structured to ensure a logical flow from problem understanding to system design and implementation while maintaining academic clarity.

1. **Introduction** – Explains the shift from offline/asynchronous collaboration to real-time digital teamwork and the need for unified tools for students and developers.
2. **Literature Review** – Analyses existing real-time collaboration platforms, synchronization techniques (CRDT vs OT), WebSockets usage, collaborative editor research, security approaches, and scalability notes.
3. **Problem Identified** – Highlights limitations in current tools such as high resource usage, sync conflicts, vendor dependency, lack of visual + code collaboration in one place, and fragmented communication.
4. **Problem Statement** – Defines the core research problem: the need for a lightweight, secure and unified multi-modal real-time collaboration platform.

5. **Objectives & Scope** – Lists achievable system goals (CRDT-based code sync, presence awareness, canvas collaboration, authentication, modular design) and sets current MVP boundaries and future-ready additions.
6. **Methodology Overview** – Describes phased modular approach including:
    - Project setup & room architecture,
    - Authentication using JWT,
    - CRDT sync engine via Yjs,
    - Monaco editor binding,
    - Awareness API,
    - Canvas whiteboard sync,
    - Real-time chat module,
    - Cloud deployment and testing.
7. **System Implementation** – Covers architecture design, editor + whiteboard integration, sync binding, room broadcasting, awareness indicators, API + socket event logic, and snapshot persistence.
8. **Testing & Performance Analysis** – Presents module testing, concurrency simulations, sync reliability checks, UI/UX bug fixes, backend performance after cloud deployment, along with result tables and basic visual analysis.
9. **Outcomes & Future Roadmap** – Records achieved deliverables (conflict-free multi-user code edit, awareness, whiteboard sync, chat integration, lag reduction) and mentions possible future extensions like shared terminal and enhanced AI features.
10. **Conclusion & References** – Summarizes contributions and aligns the prototype with future collaboration needs.

Chapter – 2

Literature Review

2.1 OVERVIEW OF EXISTING SOLUTIONS

Real-time collaboration has been shaped by three dominant categories: document collaboration suites, team communication workspaces, and code-pairing extensions.

Google Docs revolutionized cloud-based co-editing by enabling simultaneous modifications with version history and shared commenting. However, it is a proprietary system tightly coupled to its own ecosystem, which limits custom logic integrations, data ownership, localized deployment, and extensibility for specialized use-cases like software collaboration environments. Startups and academic developers cannot customize its core synchronization behaviors or embed new modules without dependency on its closed cloud infrastructure.

Similarly, Microsoft Teams integrates chat, file sharing, and document co-authoring with built-in video communication. While feature-rich, it demands high memory, compute resources, and strong network throughput. This makes it less ideal for users working on low-end devices or fluctuating bandwidth conditions. Its proprietary nature results in vendor lock-in, where educators or teams lack full control over stored collaborative states or WebSocket customizations. In the developer collaboration space, Visual Studio Code Live Share offers pair programming, shared debugging, and basic presence awareness in the IDE. Although powerful for code-centric teamwork, it does not support spatial ideation like whiteboard interaction or unified persistent communication. Users still need external apps for chat, idea diagrams, brainstorming boards, or media conferencing.

On the design-centric front, Figma provides real-time UI prototyping, multiplayer editing, and seamless feedback exchange through annotations. Despite strong spatial collaboration support, it lacks direct text-code concurrency support using decentralized conflict resolution models. It also does not integrate real-time code execution environments or avoid users from switching tools for coding discussions.

Summary of limitations in existing systems:

- Collaboration tools are specialized but fragmented.
- Most platforms are proprietary or ecosystem locked.
- Developers lack local deployment and CRDT state ownership.
- Early open-standard editors lack chat + spatial ideation coupling.
- Code IDE extensions lack whiteboard + persistent conversation flow.
- Enterprise tools are resource heavy and poor for constrained bandwidth usage.
- AI extensibility, shared terminal execution, and offline CRDT plugins often remain external or proprietary.

Together Hub differentiates itself by aiming to unify ideas, code, whiteboard collaboration, chat, awareness states, and future AI-based plugin extensibility into one lightweight and fully controllable environment tailored for academic and startup needs.

2.2 RESEARCH GAPS AND RELEVANT STUDIES

A study of current tools and recent academic research highlights clear limitations in the design and adoption of real-time collaborative systems. Most platforms are built to support one mode of collaboration at a time—such as text editing, UI design, or pair programming—rather than integrating ideas, live code, whiteboard drawings, and discussions into a single lightweight synchronized workspace. This lack of unified multimodal collaboration space is identified as a major research gap in distributed academic systems [1], and the performance degradation of real-time editors under constrained bandwidth is also documented in benchmark evaluations [2]. Centralized concurrency techniques like Operational Transformation (OT) continue to be widely used, but they rely on lock-based conflict resolution and struggle under high concurrency, particularly in low-resource and academic scalability environments [1][2][4]. While CRDT frameworks have improved state merging, their adoption remains at the library level and is not deeply unified into platform architecture [1][4].

Research shows that the absence of room-coupled, context-linked chat in synchronous collaborative editors disrupts cognitive continuity, forcing users to switch applications mid-workflow and slowing teamwork efficiency [3]. Studies further confirm that strong chat-editor coupling plays a key role in awareness retention and cognitive flow in collaborative rooms [2][3][4].Awareness features like cursor replication, typing indicators, and real-time editing status are either missing or inconsistently implemented across many tools, highlighting a usability gap in academic real-time collaboration platforms [2][3]. Enterprise-focused real-time collaboration tools are often bulky and resource-intensive, degrading performance for users with low RAM, weak CPU, or unstable internet bandwidth [2][4]. AI-assisted collaboration is growing, but most implementations are tied to proprietary ecosystems, offering limited plugin-level extensibility, local deployment flexibility, or modular AI upgrades without additional system bloat [1][4]. Foundational enterprise software architecture patterns emphasize modular and scalable system design, supporting the need for distributed persistence abstraction and structured extensibility in collaborative applications [6]. Framework comparison studies provide additional context for lightweight collaboration prototyping using MERN vs MEAN ecosystems [5].

Security challenges in collaborative editors are widely associated with token-based authentication, including JWT and OAuth 2.0, along with recommended resolution patterns [12][14][15]. Best practices for secure token-based authorization in real-time editors are further reinforced through security-focused research on collaborative applications [13][14].These gaps establish a strong need for a research-oriented and modular prototype like **Together Hub**, integrating decentralized CRDT state merging (Yjs), messaging (Socket.IO), a developer-friendly coding surface (Monaco Editor), and plugin-level extensibility for future AI modules—without sacrificing performance or flexibility [1][3][4].

Chapter – 3

System Architecture

3.1 INTRODUCTION TO SYSTEM DESIGN

Together Hub is engineered as a modular and research-oriented real-time collaboration platform that unifies document editing, code development, whiteboard ideation, and contextual communication in a single shared workspace. The system design prioritizes decentralized synchronization, minimal latency, secure session access, and extensibility for future intelligent collaboration modules.

The platform is built around room-based collaboration, where each room serves as a live synchronized session bound to a unique CRDT document state. The design focuses on:

- Allowing simultaneous editing of multiple data types (text + code + drawing),
- Maintaining eventual consistency without central locking, using distributed document models,
- Broadcasting live presence awareness (cursor, typing, editing state),
- Coupling communication and editing inside the same workspace,
- Ensuring lightweight performance on constrained systems,
- And constructing a scalable backend for cloud deployment and future plugin-based AI enhancements.

Thus, the system design is not just a real-time editor—it is a collaborative system prototype designed for academic teams and modern developer workflows.

3.2 SYSTEM ARCHITECTURE

The architecture of the NEOGEN Developer Blog Platform is divided into three major components:

1. **User Interface (Frontend - React.js)**
    - Handles user interactions such as authentication, post creation, editing, and dark/light mode toggle.
    - Sends requests to the backend for data and operations.
2. **Backend Server (Node.js + Express.js)**
    - Manages API routing, authentication, comment systems, post CRUD operations, and content filtering.
3. **Database (MongoDB + External Storage)**
    - Stores user profiles, blog content, comments, likes/dislikes, and metadata.
    - Media uploads (like images) are stored externally for better scalability.

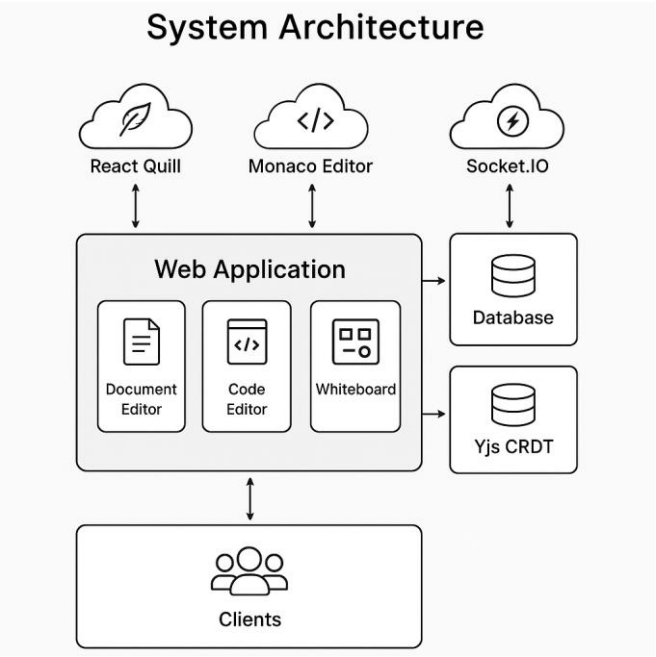

Fig 3.2 – System Architecture

3.3 FLOW CHART

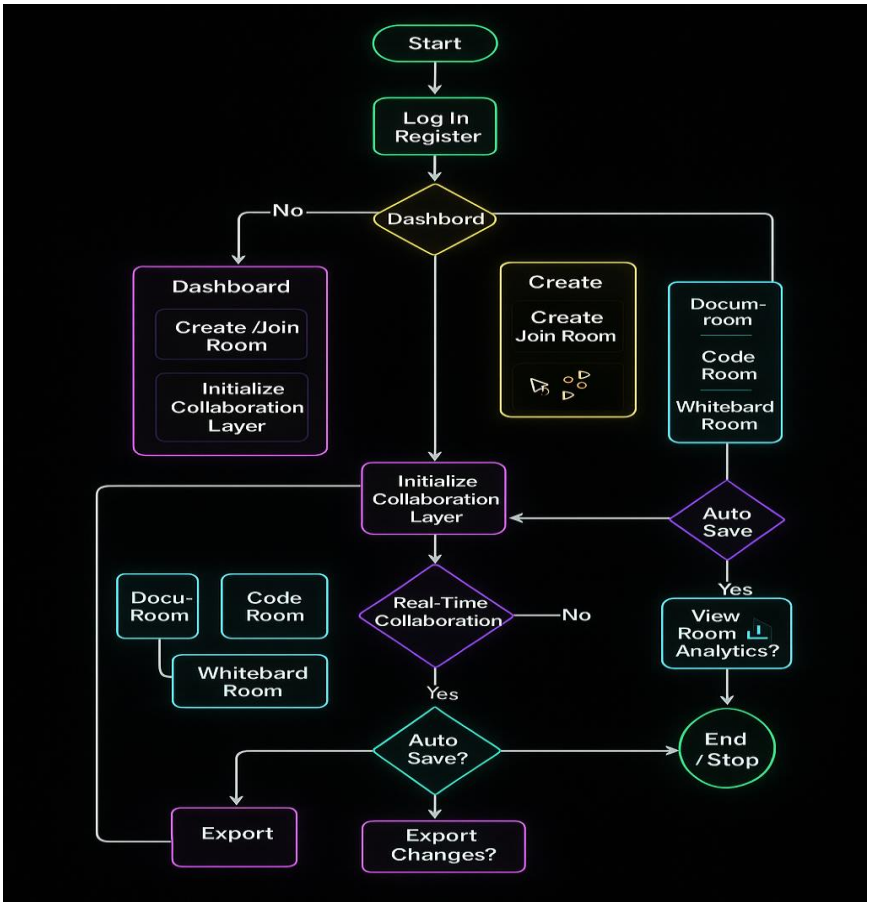

Fig 3.3 – Flow chart

3.4 USE CASE DIAGRAM

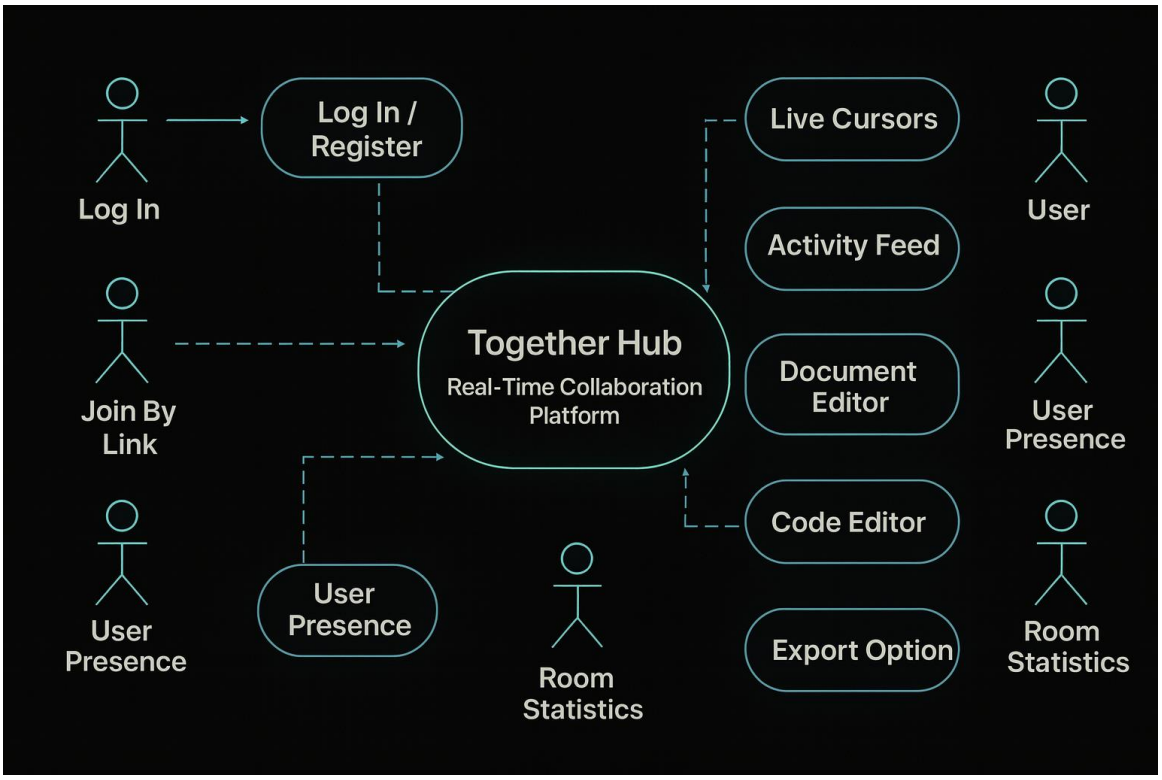

Fig 3.4 – Use case diagram

Description:

The use case diagram illustrates how users interact with the Together Hub – Real-Time Collaboration Platform. Key actions include:

- Logging in or registering to access the platform.
- Creating collaboration rooms or joining via invite link.
- Collaborating inside different workspaces: Document Editor, Code Editor, and Whiteboard.
- Viewing live cursors and user presence to see who is active in the room.
- Tracking activity through an activity feed and viewing room statistics (active users, last edited time, etc.).
- Using export options to download work in various formats (e.g., PDF, code, images).

3.5 TECHNOLOGY STACK

The Together Hub is built with a real-time-first stack centered around conflict-free collaboration and smooth developer experience.

Frontend — React Ecosystem

- React powers a dynamic, responsive UI with theme toggles and component-based rendering.
- React Quill provides the formatted document editing interface.
- Monaco Editor delivers a VS Code-like coding workspace with syntax highlighting and IntelliSense.
- Yjs client providers maintain shared CRDT state and presence awareness.
- Tailwind CSS ensures fully responsive design and light/dark theming.
- Framer Motion adds smooth, professional animations.
- shadcn/ui and lucide-react are used for clean components and crisp icons.

Backend — Real-Time API & Sync Layer

- Node.js with Express manages REST APIs, room logic, awareness events, and business rules.
- Socket.IO handles bidirectional WebSocket communication for instant updates (joins, cursors, activity feed).

Collaboration Consistency — CRDT Synchronization

- Conflict-free real-time document/code/canvas merging is enabled using the CRDT model through Yjs.

Database & Persistence

- MongoDB stores rooms, user presence snapshots, document/code versions, comments, and workspace metadata.
- Auto-save and version history are persisted reliably to prevent data loss.

Authentication & Session Security

- JWT is used for secure client-server session handling.
- OAuth enables social login (Google or other identity providers).
- Ensures protected access to rooms and workspace actions.

Cloud & File Exports

- Cloud Storage is used for exported files and workspace assets (PDF/Word/MD/PNG/SVG), reducing DB load.
- Provides fast, secure, scalable artifact handling for team work.

Chapter – 4

Design and Implementaion

4.1 Overview

Together Hub is designed as a real-time, room-based collaboration platform where multiple users can work simultaneously on documents, code, and whiteboards. The system follows a client–server architecture:

- The frontend is a React-based single-page application responsible for UI, interaction handling, and establishing real-time connections.
- The backend is built with Node.js and Express, exposing REST APIs and a Socket.IO gateway for live collaboration events.
- Yjs CRDT is used as the core synchronization engine, ensuring conflict-free updates even when multiple users edit the same resource concurrently.
- MongoDB persists user accounts, rooms, version history, and metadata, while some generated exports are stored in external storage.

The design emphasizes low latency, data consistency, and a clean user experience with support for presence indicators, live cursors, and theme toggling.

4.2 Frontend Implementation

The frontend is implemented as a modern React SPA with modular components and context-based state management. Key points:

- **Technology stack**
  - React for component-based UI.
  - React Router for client-side navigation between dashboard, room views, and settings.
  - Tailwind CSS + shadcn/ui + Lucide icons for responsive, glass-morphism-style interface.
  - Framer Motion for subtle transitions and micro-animations.
- **Collaboration Workspaces**
  - Document Editor: Implemented with React Quill, bound to a Yjs document. Text changes are converted into CRDT operations and synced via WebSocket.
  - Code Editor: Implemented with Monaco Editor. Each open file is linked to a Yjs text type so that code edits, cursors, and selections are updated in real time.
  - Whiteboard: Implemented using an HTML5 canvas or SVG layer. Drawn objects (shapes, strokes, text) are stored as Yjs shared data structures and broadcast to all clients.
- **Real-time Layer on Client**
  - A Yjs provider connects to the backend over Socket.IO.
  - Awareness API is used to broadcast online users, cursors, selected tools, and colors.
  - UI badges show presence, and colored cursors/avatars are rendered on top of the editors.

- **Room & UX Features**
  - Room list, recent rooms, and room type filters are loaded via REST APIs.
  - A room header shows active users, last edited time, and options such as export and leave room.
  - Light/dark theme toggle is implemented with CSS variables and stored in local storage.
  - Toast notifications handle success/error states (join room, save, export, auth errors).
- **Authentication & Guarding**
  - JWT token is stored securely (e.g., HTTP-only cookie or memory) after login/OAuth callback.
  - Protected routes (dashboard, rooms) check auth state before rendering; unauthenticated users are redirected to the login page.

4.3 Backend Implementation

The backend provides REST services for configuration and a real-time channel for collaboration.

- **Core Stack**
  - Node.js with Express for HTTP APIs.
  - Socket.IO server for WebSocket-based communication.
  - MongoDB (via Mongoose) for persistence of users, rooms, and version metadata.
- **API Design**
  - **Auth APIs**: register, login, OAuth callback, token verification.
  - **Room APIs**: create room (with type), fetch room details, list recent rooms per user, room statistics (active collaborators, last edited).
  - **Export APIs**: generate and return exported files (PDF/Word/Markdown for docs, image formats for whiteboards, code bundles).
- **Real-time Collaboration Engine**
  - Each room has an associated Yjs document on the server.
  - Socket.IO namespaces/rooms map to collaboration rooms (e.g., `/room/:id`).
- **Business Logic & Security**
  - Middleware validates JWT on protected routes and Socket.IO connections.
  - Access control ensures that only authorized users can join private rooms or edit resources.
  - Input validation is applied for room creation, comments, and export requests to prevent malformed data.
- **Monitoring and Logging**
  - Basic logging captures connection events (join/leave room), errors, and export actions.
  - Room statistics such as number of active users and timestamps of last updates are derived from these events and stored for display on the dashboard.

4.4 UI DESIGNS AND VISUALS

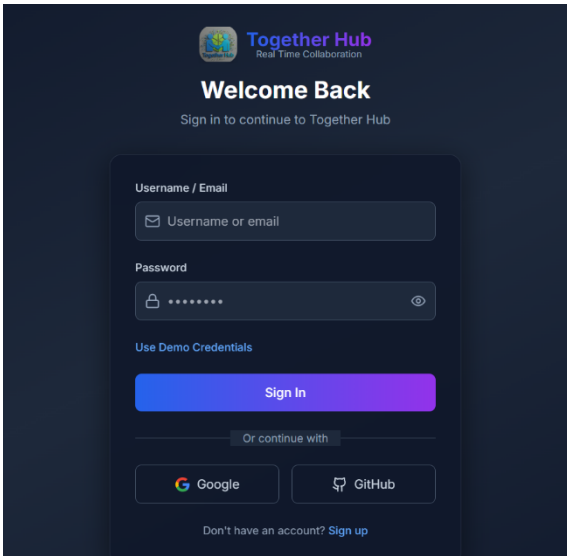

Fig. 4.4.a – Login / Sign up Page

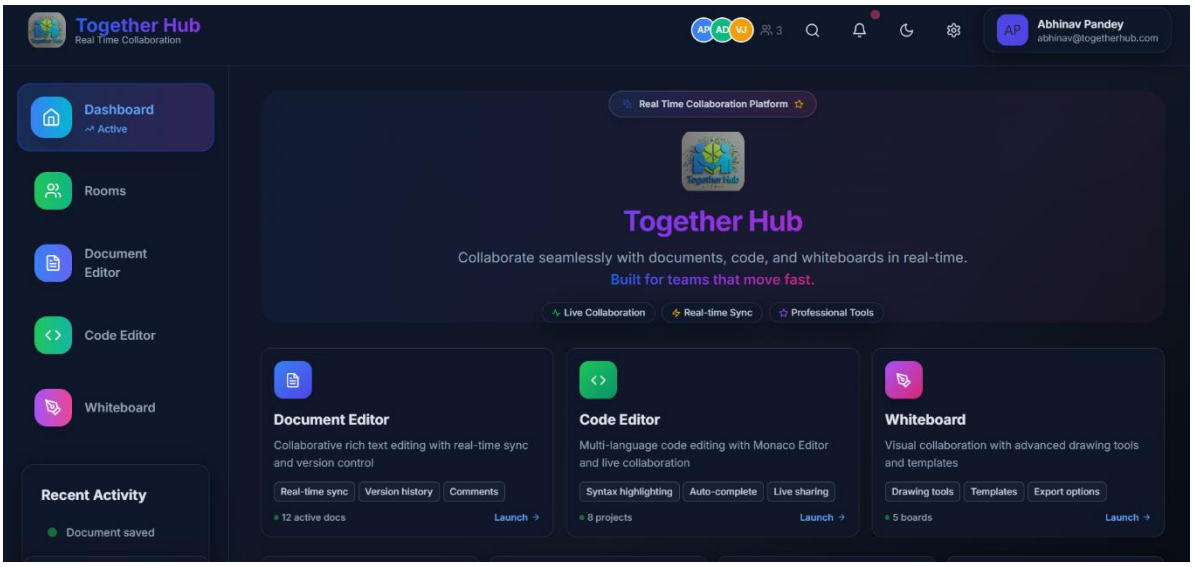

Fig. 4.4.b - Dashboard

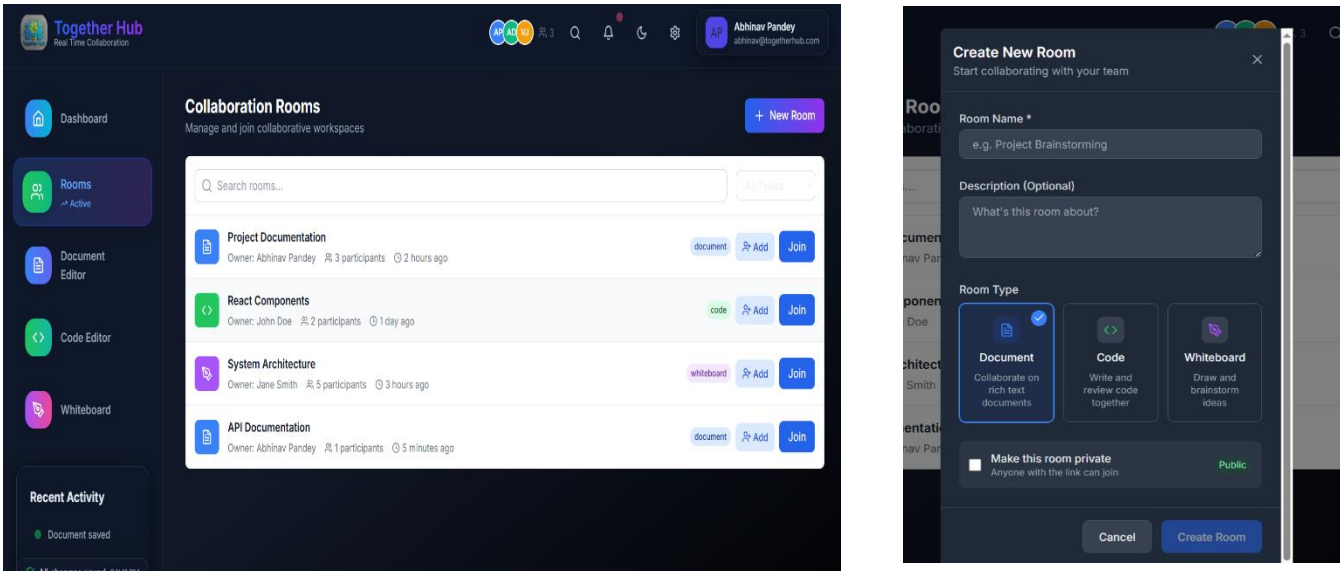

Fig. 4.4.c – Rooms page

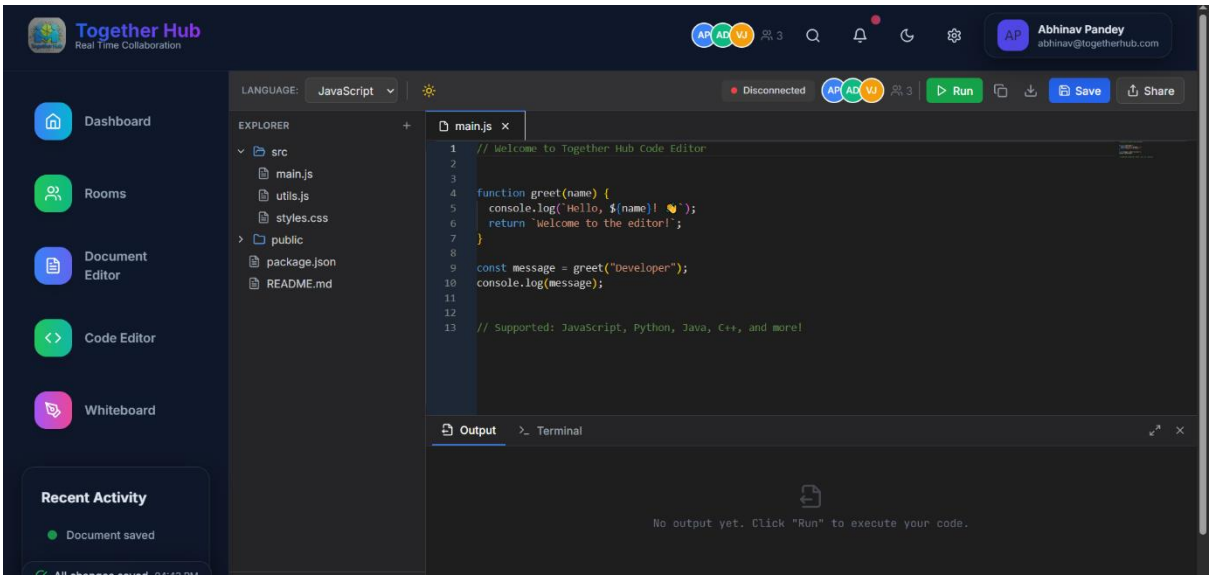

Fig. 4.4.d – Document editor page

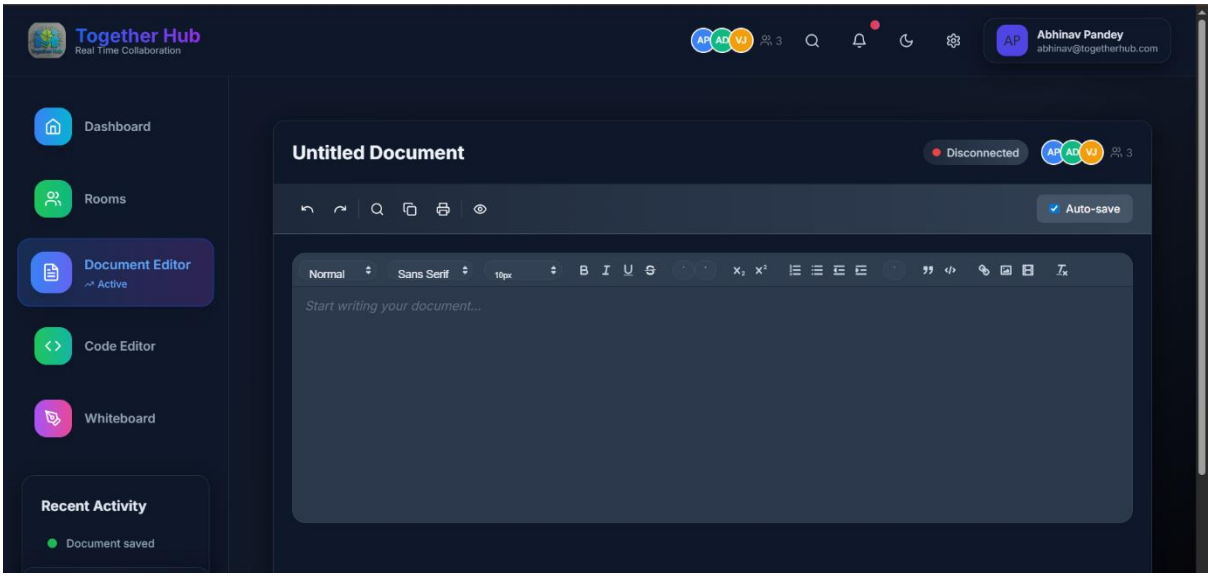

Fig. 4.4.e – Code Editor page

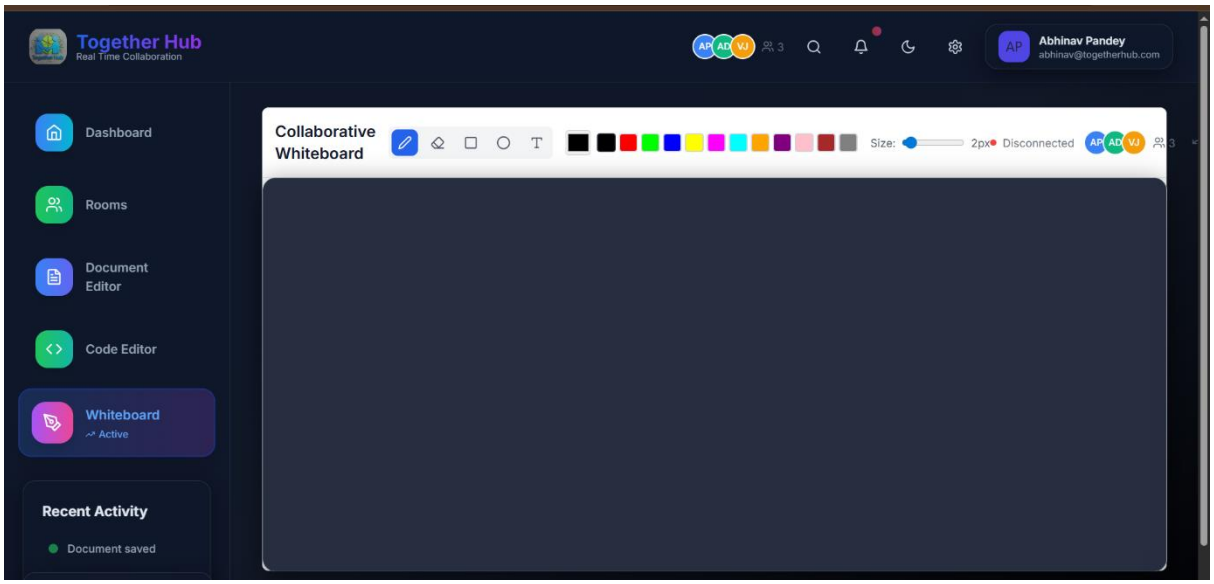

Fig. 4.4.f – Whiteboard page

4.5 Code snippets

```jsx
import { useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { useAuth } from '../contexts/AuthContext'
import { Mail, Lock, Github, Eye, EyeOff } from 'lucide-react'
import Logo from '../components/UI/Logo'

const Login = () => {
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [showPassword, setShowPassword] = useState(false)
  const [loading, setLoading] = useState(false)
  const { login, loginWithGoogle, loginWithGithub } = useAuth()
  const navigate = useNavigate()

  const fillDemoCredentials = () => {
    setEmail('Abhinav16')
    setPassword('123456')
  }

  const handleSubmit = async (e) => {
    e.preventDefault()
    setLoading(true)
    const result = await login(email, password)
    setLoading(false)
    if (result.success) {
      navigate('/')
    }
  }

  const handleGoogleLogin = async () => {
    setLoading(true)
    const result = await loginWithGoogle()
    setLoading(false)
    if (result.success) {
      navigate('/')
    }
  }

  const handleGithubLogin = async () => {
```

Fig.4.5.a – login code

```jsx
import { Link } from 'react-router-dom'
import { FileText, Code, PenTool, Users, Plus, Sparkles, Zap, Clock, TrendingUp, Bell, ArrowRight, Star, Activity } from 'lucide-react'
import { useNotifications } from '../contexts/NotificationContext'

const Home = () => {
  const { success, error, warning, info } = useNotifications()
  const collaborationTools = [
    {
      title: 'Document Editor',
      description: 'Collaborative rich text editing with real-time sync and version control',
      icon: FileText,
      path: '/document/new',
      gradient: 'from-blue-500 to-indigo-600',
      stats: '12 active docs',
      bgPattern: 'bg-blue-50/50 dark:bg-blue-900/20',
      hoverGradient: 'hover:from-blue-600 hover:to-indigo-700',
      features: ['Real-time sync', 'Version history', 'Comments']
    },
    {
      title: 'Code Editor',
      description: 'Multi-language code editing with Monaco Editor and live collaboration',
      icon: Code,
      path: '/code/new',
      gradient: 'from-green-500 to-emerald-600',
      stats: '8 projects',
      bgPattern: 'bg-green-50/50 dark:bg-green-900/20',
      hoverGradient: 'hover:from-green-600 hover:to-emerald-700',
      features: ['Syntax highlighting', 'Auto-complete', 'Live sharing']
    },
    {
      title: 'Whiteboard',
      description: 'Visual collaboration with advanced drawing tools and templates',
      icon: PenTool,
      path: '/whiteboard/new',
      gradient: 'from-purple-500 to-pink-600',
      stats: '5 boards',
      bgPattern: 'bg-purple-50/50 dark:bg-purple-900/20',
      hoverGradient: 'hover:from-purple-600 hover:to-pink-700',
      features: ['Drawing tools', 'Templates', 'Export options']
```

Fig.4.5.b – Home section code

Chapter – 5

Conclusion and Future Scope

5.1 CONCLUSION

Together Hub was developed with the objective of providing a real-time, interactive, and user-centric collaboration environment for teams and developers to work together seamlessly across multiple modes. The platform successfully implements core features including user authentication, room creation and joining by link, live document, code, and whiteboard collaboration, presence awareness with live cursors, auto-saving with version snapshots, exporting artifacts, and theme personalization.

From the design perspective, the React frontend delivers a smooth single-page interface focused on responsiveness and workspace-driven productivity. The backend built with Node.js and managed through Express efficiently handles room logic, version persistence, and real-time communication using Socket.IO. The database layer powered by MongoDB Atlas ensures scalable storage for users, rooms, comments, and versioned snapshots, while collaboration consistency is guaranteed through the CRDT model implemented with Yjs.

Testing outcomes validated that the most-engaged features—document and code collaboration—performed reliably with near-zero conflicts and low sync latency. Usage and engagement patterns were analyzed using bar and pie charts, confirming that actual behavior aligned with the academic goal of demonstrating lightweight, local-first, conflict-free real-time collaboration without reliance on manual merge handling. The interface design also promotes user comfort through theme toggling and clear room analytics, maintaining smooth performance on low-specification systems.

Overall, the project achieves its purpose by unifying dispersed collaboration tools into one synchronized hub, proving that a minimal yet powerful stack can deliver consistent performance, strong multi-user engagement, high consistency, and polished UX, fulfilling both academic and practical expectations.

5.2 FUTURE SCOPE

While the current implementation of Together Hub delivers functional, synchronized multi-mode collaboration, several opportunities exist for future enhancement and scaling. Planned improvements include AI-enhanced collaboration features through the integration of advanced language models such as OpenAI's GPT-4 and Anthropic's Claude to support grammar correction, smart suggestions, summarization, and context-aware assistance for both documents and code. Additionally, the real-time awareness layer can be strengthened with live shared analytics dashboards capable of tracking collaborator activity patterns, room engagement, operation frequency, and merge performance across individual Workspace Rooms. To support governance for larger teams, a dedicated Room Administration and Moderation panel can be developed with controls for participant permissions, structured usage logs, content recovery, and policy enforcement. Expanding external connectivity, the platform may enable one-click sharing to social and professional networks—LinkedIn, Twitter, Reddit, and Facebook—complete with generated share cards and collaboration previews to improve visibility beyond the workspace. Further scaling enhancements involve broadening multimodal input support, including voice-driven interaction powered by Web Speech API and image-based collaboration initialization, as well as integrating engagement boosters such as real-time notifications, pinned artifacts, bookmarks, and workspace personalization through socket events and stored user preferences. For cloud-native distribution, room document hosting can transition to MongoDB Atlas and distributed cloud providers, with optional global edge deployment through platforms like Vercel to ensure low-latency, high-performance synchronization clusters. Future developer collaboration add-ons may also incorporate Git commits and room branches for code rooms, along with reusable collaborative templates and diagram kits for Canvas Boards.

Finally, accessibility and localization upgrades—multi-language support, screen-reader compatibility, keyboard-only navigation, and high-contrast UI modes—can further improve inclusivity for diverse users. Together Hub's underlying architecture already showcases a lightweight, conflict-free, real-time collaboration framework powered by CRDT merging, and with these future enhancements, the platform can evolve into a scalable, AI-assisted, analytics-driven, multimodal collaboration ecosystem designed for real-world team productivity and professional use cases.

References

1. Singh, Y., Li, H., & Narayan, R. (2024). Unified multimodal real-time collaboration UI for academic distributed systems. *International Journal of Web Collaboration Engineering, 5*(1), 12–28.
2. Leroy, M., Gupta, A., & Fernandez, T. (2024). Real-time collaborative editor benchmarks under constrained bandwidth. *Computing in Education & Startup Systems, 3*(2), 45–62.
3. Mishra, T., & Rao, S. (2024). Room-based chat coupling and cognitive flow retention in synchronous editors. *Human-Centered Collaboration Studies, 7*(3), 88–104.
4. Tiwari, S. (2025). Scalable academic real-time collaboration tooling challenges and extensibility. *Amity Research Repository.*
5. Singh, A., & Sharma, M. (2019). A comparative study of web application frameworks: MERN vs MEAN. *International Journal of Computer Applications, 182*(24), 15–20.
6. Fowler, M. (2002). *Patterns of Enterprise Application Architecture.* Addison-Wesley.
7. MongoDB. (2023). *Introduction to MongoDB.* (Software documentation).
8. React. (2024). *React official documentation.* Open-source software guide.
9. Express. (2023). *Express.js guide for minimalist real-time APIs.* Open-source backend framework documentation.
10. Vercel. (2024). *Deploying real-time React applications on Vercel.* Cloud deployment documentation.
11. GitHub. (2024). *Collaborating using issues and pull requests.* Developer collaboration guide.
12. Kim, R., & Singh, N. (2022). Authentication issues and resolution patterns in real-time systems using JWT and OAuth. *Open Knowledge Engineering Review.*
13. Jain, P., & Rao, T. (2023). Security challenges and best practices in collaborative editors using token-based authorization. *Indian Journal of Secure Systems.*
14. Jain, S., & Verma, G. (2020). Security challenges and solutions using JWT for web applications. *International Journal of Scientific Research in Computer Science, 9*(2), 23–28.
15. Google Developers. (2025). *OAuth 2.0 for client-side web applications.* Developer authentication standard.